

# Module 4: Kubernetes Hands-On with the NSF Nautilus HyperCluster

GLDA 4003: Cloud Computing for Data Analytics



University of Missouri

# Part 1: Kubernetes Hands-On



# Yet Another Markup Language (YAML)

| XML                                                                                                                                                                                                                                       | JSON                                                                                                                            | YAML                                                                                 |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| <pre>&lt;Servers&gt;   &lt;Server&gt;     &lt;name&gt;Server1&lt;/name&gt;     &lt;owner&gt;John&lt;/owner&gt;     &lt;created&gt;123456&lt;/created&gt;     &lt;status&gt;active&lt;/status&gt;   &lt;/Server&gt; &lt;/Servers&gt;</pre> | <pre>{   Servers: [     {       name: Server1,       owner: John,       created: 123456,       status: active     }   ] }</pre> | <pre>Servers: - name: Server1   owner: John   created: 123456   status: active</pre> |

- ▶ YAML is a key-value pair file format, similar to JSON and XML
- ▶ Kubernetes operations are performed using **YAML** files, known as a Spec file
  - ▶ Creating Persistent Storage
  - ▶ Creating Pods
  - ▶ Creating Jobs
  - ▶ Deploying services



# Synopsis of YAML language<sup>1</sup>

- ▶ Comments in YAML begins with the (#) character and they must be separated from other tokens by whitespaces.
- ▶ Indentation of whitespace is used to denote structure.
- ▶ Tabs are not included as indentation for YAML files.
- ▶ Lists are important
  - ▶ List members are denoted by a leading hyphen (-).
  - ▶ List members are enclosed in square brackets and separated by commas.
  - ▶ YAML always requires colons and commas used as list separators followed by space with scalar values.
- ▶ Associative arrays are represented using colon ( : ) in the format of key value pair. They are enclosed in curly braces {}.



# Structure of YAML file

- ▶ Each YAML file consists of specific parts, each part is dedicated to hold a type of information that enables us to communicate our needs to the Kubernetes cluster.
- ▶ Kind - tells Kubernetes what the type of the object: pod, job, service
- ▶ metadata - the main thing to specify is the name
- ▶ Spec - the attributes specified in this section depends on the kind
- ▶ There are common spec attributes



# Config file (YAML)

## Pod without mounted volume

We begin by setting the API Version and the type of object we are creating (Pod), as well as the name of the pod

From here we are defining the container to run in this pod

Set the name of the container, the image the container should run, and the command that should run when the container begins

Here, we define the requested and maximum amount of resources our container needs to run, in this case that is 2 CPU cores and 4 GB of RAM

```
apiVersion: v1
kind: Pod
metadata:
  name: python3-pod
spec:
  containers:
    - name: python3-container
      image: python:3.8
      command: ["sleep", "infinity"]
      resources:
        limits:
          memory: 4Gi
          cpu: 2
        requests:
          memory: 4Gi
          cpu: 2
```



# Config file (YAML)

## Job without mounted volume

We begin by setting the API Version and the type of object we are creating (Job), as well as the name of the job

From here we are defining the pod that is to be started by this job

Set the name of the container, the image the container should run, and the command that should run when the container begins

Here, we define the requested and maximum amount of resources our container needs to run, in this case that is 2 CPU cores and 4 GB of RAM

```
apiVersion: v1
kind: job
metadata:
  name: python3-job
spec:
  template:
    containers:
      - name: python3-pod
        image: python:3.8
        command: ["sleep", "infinity"]
        resources:
          limits:
            memory: 4Gi
            cpu: 2
          requests:
            memory: 4Gi
            cpu: 2
```



# Config file (YAML)

## Persistent volume (PVC)

We begin by setting the API Version and the type of object we are creating (PersistentVolumeClaim), as well as the name of the PersistentVolumeClaim

From here we are defining the class of the persistent volume we are using

Here we define the type of access mode of the persistent volume we are creating

Here, we define the requested size of the persistent volume 50 GB of storage

To increase the size of the persistent volume storage we only need to modify this value  
We can only increase but we cannot decrease

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: your_name
spec:
  storageClassName: rook-cephfs
  accessModes:
    - ReadWriteMany
  resources:
    requests:
      storage: 50Gi
```



# Config file (YAML)

## Pod with mounted volume

We begin by setting the API Version and the type of object we are creating (Pod), as well as the name of the pod

From here we are defining the container to run in this pod

Set the name of the container, the image the container should run, and the command that should run when the container begins

Here, we define the requested and maximum amount of resources our container needs to run, in this case that is 2 CPU cores and 10 GB of RAM

Information of the mounted volume and how it is defined within the pod

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-name-sso
spec:
  containers:
    - name: pod-name-sso
      image: python:3.8
      command: ["sh", "-c", "echo 'Im a new pod'
&& sleep infinity"]
      resources:
        limits:
          memory: 12Gi
          cpu: 2
        requests:
          memory: 10Gi
          cpu: 2
      volumeMounts:
        - mountPath: /data
          name: anes-pv
  volumes:
    - name: anes-pv
      persistentVolumeClaim:
        claimName: anes-pv
```



# Config file (YAML)

## Job with mounted volume

We begin by setting the API Version and the type of object we are creating (Job), as well as the name of the job

From here we are defining the pod that is to be started by this job

Set the name of the container, the image the container should run, and the command that should run when the container begins

Here, we define the requested and maximum amount of resources our container needs to run, in this case that is 2 CPU cores and 4 GB of RAM

Information of the mounted volume and it is defined within the job

```
apiVersion: v1
kind: job
metadata:
  name: python3-job
spec:
  template:
    containers:
      - name: python3-pod
        image: python:3.8
        command: ["sleep", "infinity"]
        resources:
          limits:
            memory: 4Gi
            cpu: 2
          requests:
            memory: 4Gi
            cpu: 2
        volumeMounts:
          - name: canada
            mountPath: /Canada
    volumes:
      - name: canada
        persistentVolumeClaim:
          claimName: canada2019-3
```



# Interfacing with Kubernetes: KubeCTL

- ▶ With a published Docker image and prepared YAML Spec file, KubeCTL enables interaction with Kubernetes:

```
kubectl [command] [TYPE] [NAME] [flags]
```

where:

- ▶ **command**: Specifies the operation that you want to perform on one or more resources, for example create, get, describe, delete
- ▶ **TYPE**: Specifies the resource type, such as pod or job
- ▶ **NAME**: Specifies the name of the resource, or the path to a Spec file
- ▶ **flags**: Specifies optional flags, such as `--server` to specify the address and port of the API server



# Interfacing with Kubernetes: KubeCTL cheat sheet

## ► To create pod

```
kubectrl create -f pod.yaml
```

```
kubectrl apply -f pod.yaml
```

## ► To create job

```
kubectrl create -f job.yaml
```

```
kubectrl apply -f job.yaml
```



# Interfacing with Kubernetes: KubeCTL cheat sheet

## ► To check pod status

```
kubectl get pods
```

```
kubectl describe pod pod-name
```

## ► To check job status

```
kubectl get jobs
```

```
kubectl describe job job-name
```

## ► To debug pod

```
kubectl logs pod-name
```



# Interfacing with Kubernetes: KubeCTL cheat sheet

## ► To delete pod

```
kubectrl delete -f pod.yaml
```

```
Kubectrl delete pod-name
```

## ► To delete job

```
kubectrl delete -f job.yaml
```

```
Kubectrl delete job-name
```



# Interfacing with Kubernetes: KubeCTL cheat sheet

## ► To create persistent volume

```
kubectl create -f pvc.yaml
```

```
Kubectl apply -f pvc.yaml
```

## ► To increase the size of persistent volume

```
kubectl apply -f pvc.yaml
```

## ► To delete persistent volume

```
kubectl delete -f pvc.yaml
```

```
kubectl delete pvc-name
```



# **Part 2:**

## **National Research Platform**

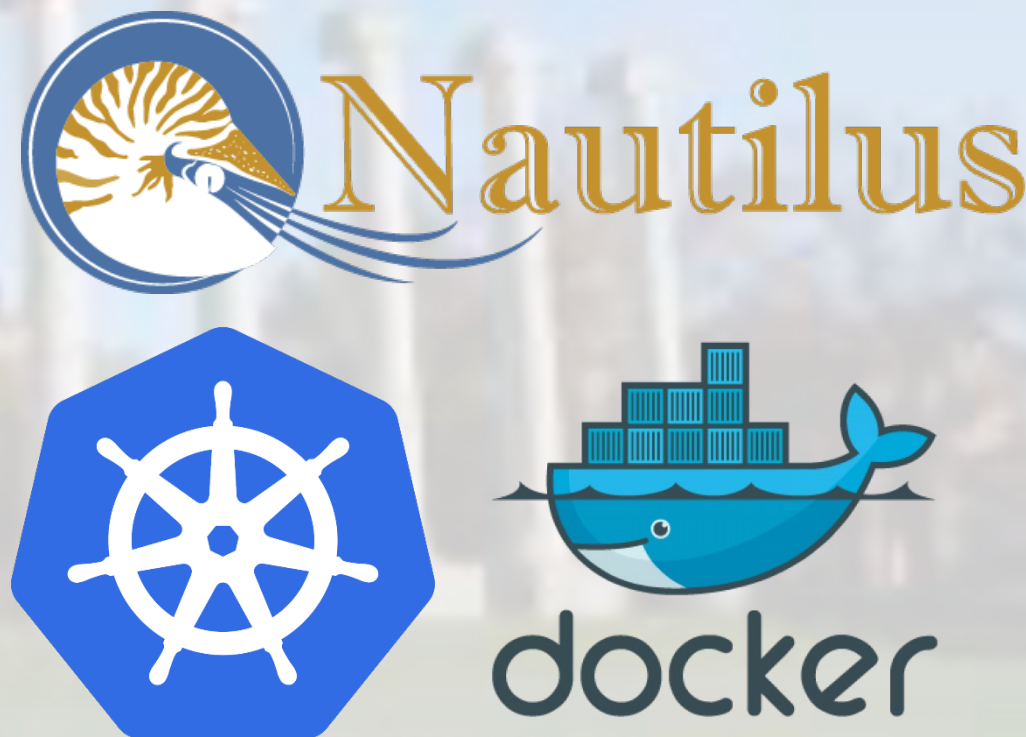
### **Nautilus Hyper Cluster**



**University of Missouri**

# NSF NRP Nautilus HyperCluster

- ▶ The NSF Nautilus HyperCluster is a Kubernetes cluster with vast resources that can be utilized for various research purposes:
  - ▶ Prototyping research code
  - ▶ S3 cloud storage for data and models
  - ▶ Accelerated small-scale research compute
  - ▶ Scaling research compute for large scale experimentation
- ▶ Resources Available:
  - ▶ CPU Cores: 14,462
  - ▶ RAM: 69 TB
  - ▶ NVIDIA GPUs: 1150



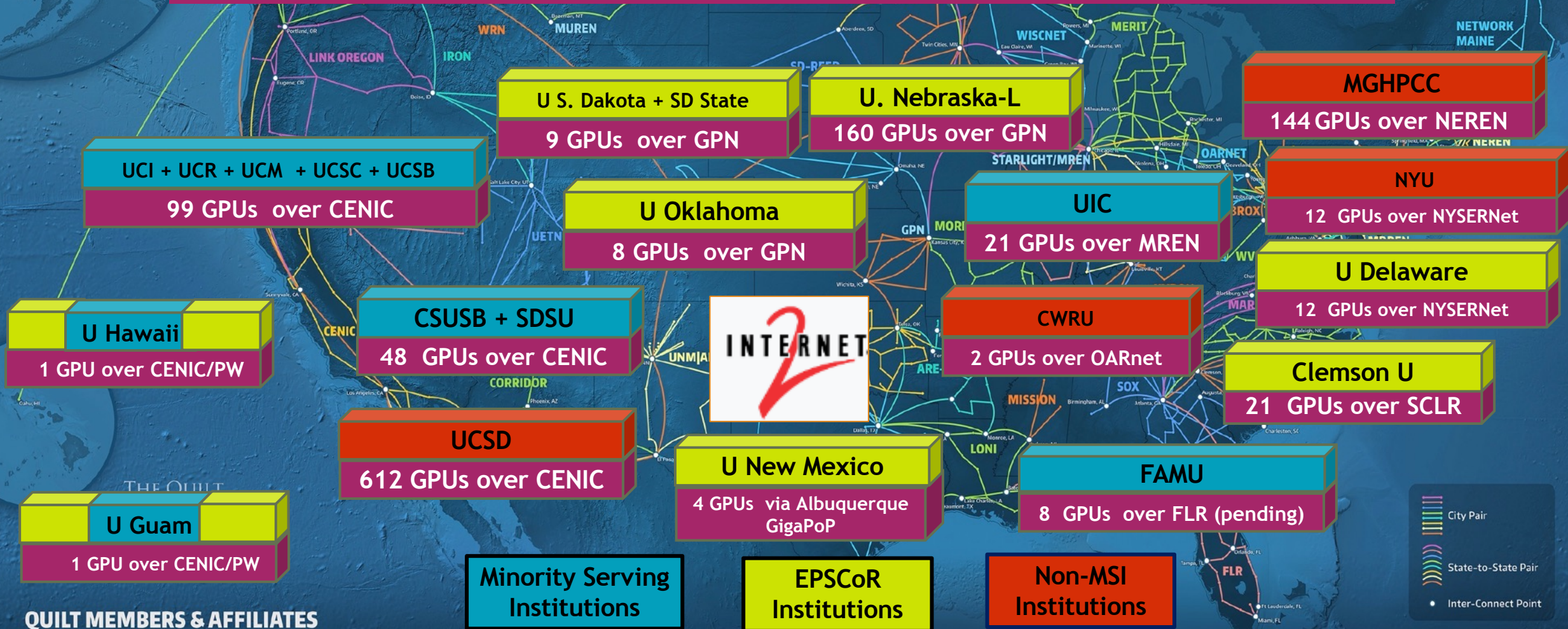
**University of Missouri**

Nautilus Logo: <https://portal.nrp-nautilus.io/>

Docker Logo: <https://developers.redhat.com/blog/2014/05/15/practical-introduction-to-docker-containers>

Kubernetes Logo: [https://commons.wikimedia.org/wiki/File:Kubernetes\\_logo\\_without\\_workmark.svg](https://commons.wikimedia.org/wiki/File:Kubernetes_logo_without_workmark.svg)

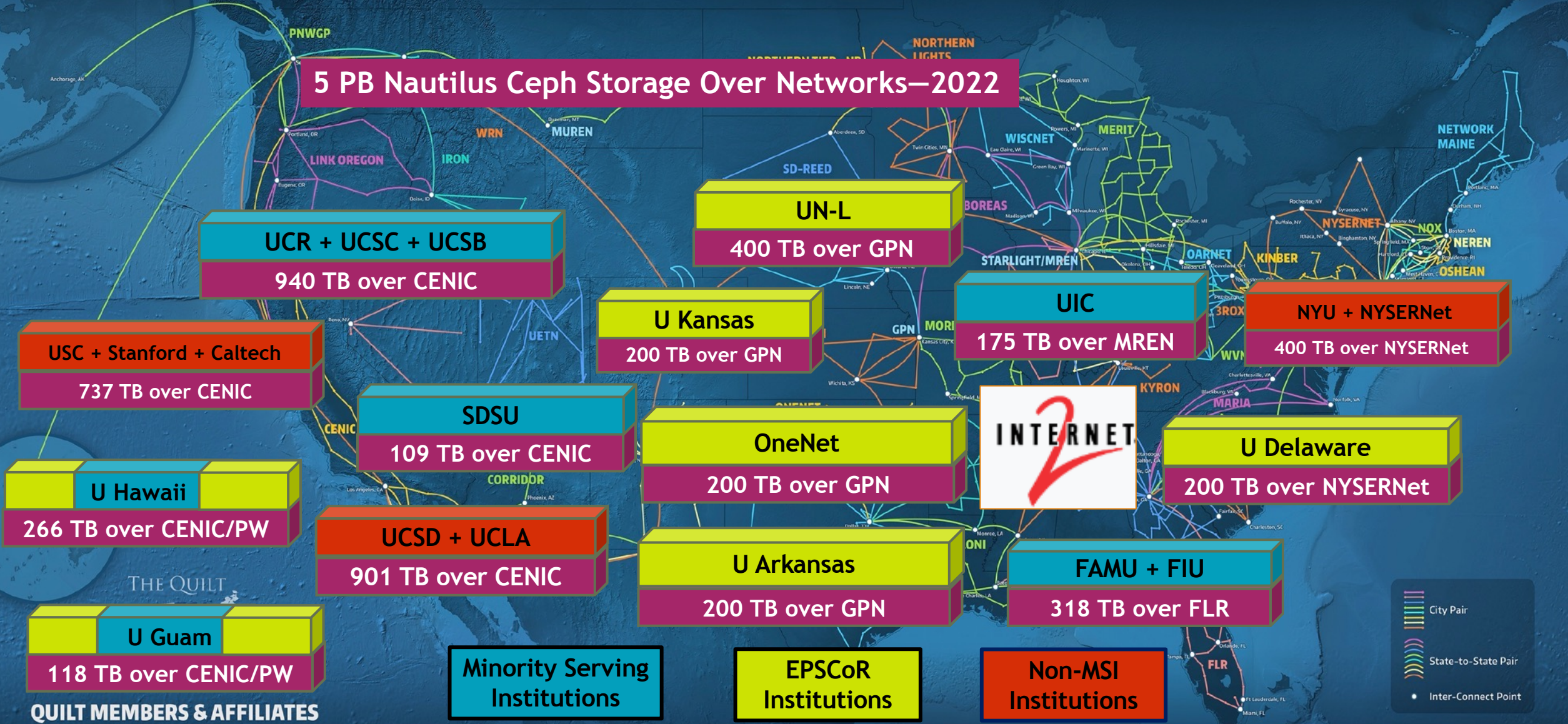
# Nautilus ~1,100 GPUs Distributed over US Networks—Fall 2022



## QUILT MEMBERS & AFFILIATES



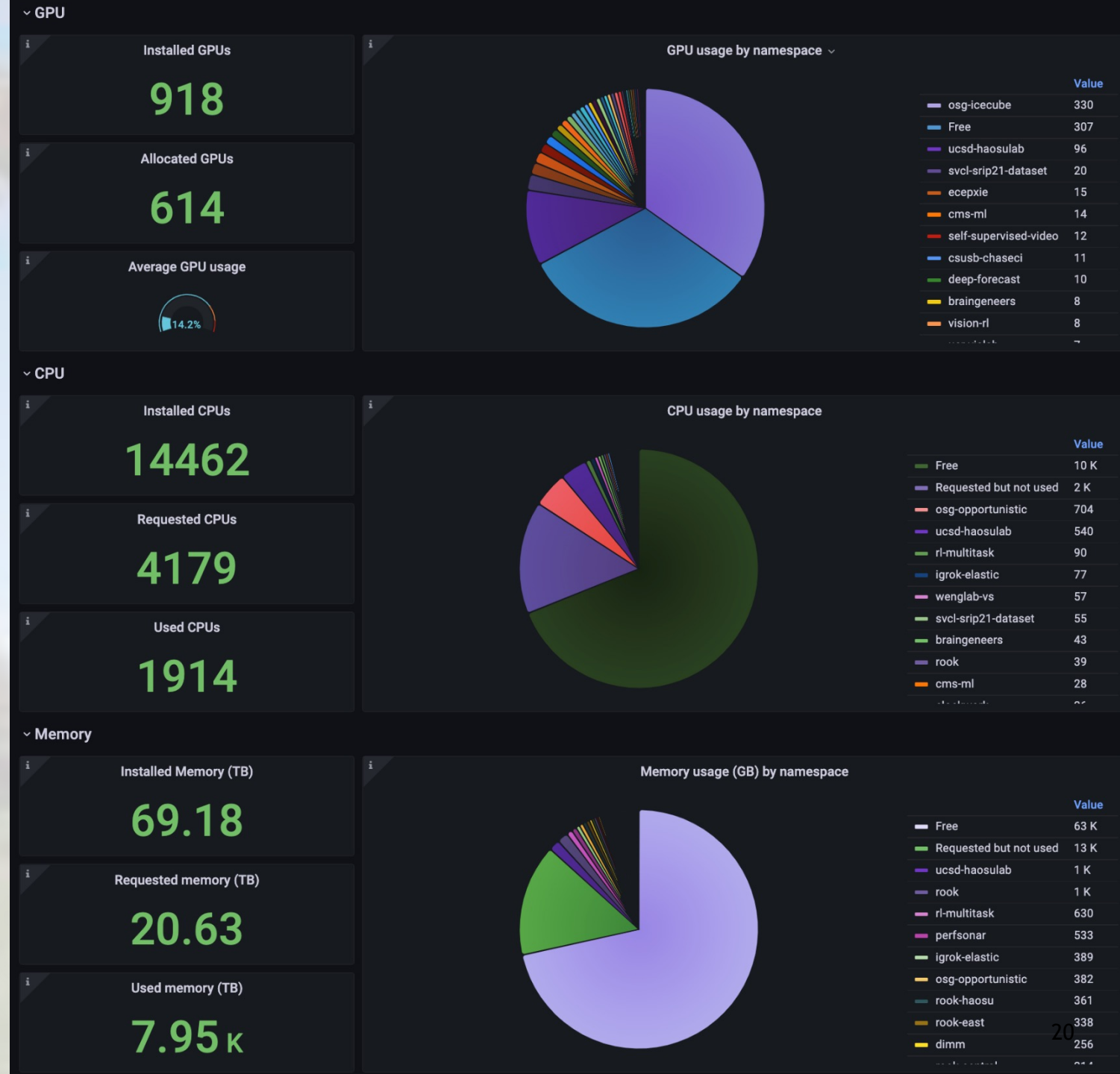
# 5 PB Nautilus Ceph Storage Over Networks—2022



# Resource Dashboard



University of Missouri



# How MU is using Nautilus: Teaching & Research

- ▶ JupyterHub integration enables creation of an interactive learning environment for STEM Education:
  - ▶ Data Science
  - ▶ Mathematics
  - ▶ High Performance Computing
- ▶ Access to vast amount of RAM, CPU Cores, and NVIDIA GPUs can accelerate research in various fields:
  - ▶ Bioinformatics
  - ▶ Remote Sensing
  - ▶ Materials Science
  - ▶ Computer Vision
  - ▶ Machine Learning



# Access to Nautilus

- ▶ Follow the steps in getting started
  - ▶ <https://ucsd-prp.gitlab.io/userdocs/start/get-access/>
- ▶ Step 1: Access Nautilus portal
- ▶ Step 2: Click on login



Namespaces overview Resources **Login**

## NRP Kubernetes portal

Here you can get an account in National Research Platform kubernetes portal by logging in with your university's credentials and requesting access in [matrix]

Documentation: <https://docs.nationalresearchplatform.org/>

You can easily join your node in our cluster - request instructions in [matrix] #general channel.

The National Research Platform currently has no storage that is suitable for HIPAA, PID, FISMA, FERPA, or protected data of any kind. Users are not permitted to store such data on NRP machines.

# Access to Nautilus

- ▶ Follow the steps in getting started
  - ▶ <https://ucsd-prp.gitlab.io/userdocs/start/get-access/>
- ▶ Step 3: Find the CILogon page
- ▶ Step 4: Select identity provider (GitHub and Google are available)



Consent to Attribute Release ▼

Nautilus requests access to the following information. If you do not approve this request, do not proceed.

- Your CILogon user identifier
- Your name
- Your email address
- Your username and affiliation from your identity provider

Select an Identity Provider

University of Missouri System ▼ ⓘ

☐ Remember this selection ⓘ

Log On

By selecting "Log On", you agree to the [privacy policy](#).

# Access to Nautilus

- ▶ Follow the steps in getting started
  - ▶ <https://ucsd-prp.gitlab.io/userdocs/start/get-access/>
- ▶ Step 5: Added to a namespace
  - ▶ Email needs to be visible

